CMSC 25422 Machine Learning for Computer Systems

Final Project Report - Reproducing netML VPN-nonVPN Application Identification

Mariska Kassi, Natalie Gergov, Longyang Li

Introduction

Identifying VPN traffic in encrypted network flows is a critical challenge in network security and management. When users employ VPNs, their traffic is encrypted, making it difficult to understand what activities are taking place. The ability to distinguish VPN from non-VPN traffic enables better network management, security monitoring, and policy enforcement. Our primary goal is research and reproducibility: to replicate and extend previous results by testing additional algorithms and comparing performance metrics.

This project builds upon the NetML and nPrint frameworks, which provide standardized benchmarks for machine learning applications in networking [1]. The nPrint VPN-NonVPN benchmark establishes baseline performance metrics using flow-based feature extraction.

Exploratory Data Analysis

The original dataset, traffic.pcapng, with its corresponding metadata, in metadata.csv, we could associate each packet to various applications (i.e. “video_vimeo_video” with Vimeo website streaming). Since there was no clear VPN/non-VPN label, we looked into the trace to label the data ourselves.

We looked into the metadata labels and noticed that some were labeled with `tor`. Looking it up, we found that that keyword could stand for “The Onion Router,” a network overlay that anonymizes network communication, similar to a VPN [2]. With this in mind, we labeled all flows with packets that contained the word ‘tor’ as VPN traffic (True Label), and all flows with no packets containing the word ‘tor’ as non-VPN traffic (False Label).

Although this was the labeling strategy we decided to go with. We noticed some possible disadvantages. First, there was an extreme imbalance between the number of VPN-labeled flows/packets and non-VPN-labeled flows/packets. This could result in the models not having enough data to detect VPN traffic due to the overwhelming amount of non-VPN samples. At this point of data processing, we could already foresee an issue with false positives and low precision across all models. Secondly, besides the issues with class imbalance (which shall be discussed later on when reviewing results), the models are limited to detecting Tor VPN traffic, not necessarily all VPN traffic in general. They might pick up specific characteristics or timing patterns specific to Tor traffic and thus might fail on other VPNs. We would need to expand our data—and specifically our VPN datapoints—in order to get a more generalizable, more robust model.

Data Preprocessing and Feature Selection

We used IAT and STATS as our features and used the netML library as our way of extracting these features from the trace data: the IAT features capture inter-arrival times between packets, and the STATS features capture the distribution and statistical features of packet sizes.

Model Training and Evaluation

Using the Sci-Kit Learn Python packages, we trained three models: Random Forest, Naive Bayes, and Lasso Regression, to classify non-VPN and VPN data.

To evaluate and compare each model, we calculated each model's Accuracy, F1 score, Precision, Recall, and ROC-AUC. In addition, we also illustrated each model's confusion matrices to clearly determine True & False positives and True and False Negatives, and illustrated all the models' ROC curves. The exact value for all of these metrics can be found on the Python notebook. Graphs depicting and comparing these values are featured in Figures #1-5 at the end of this document.

Out of all three models, the Naive Bayes Model performed the best as it achieved the highest out of the three in F1 score, Precision, Recall, and ROC-AUC.

Discussion and Conclusion

The models performed poorly in distinguishing between VPN/non-VPN data. Although the model accuracy was high, the most informative metrics here are the precision: although the models catch most of the VPN instances, they also predict thousands more false positives, decreasing precision. Thus, these models are not useful when it comes to utilizing the information that something is a VPN, as when an application wants to block/stop/know when something is a VPN, there are too many false positives produced by these models to do so. The high accuracy stems from the high class imbalance, as 99.6% of the data belongs to the non-VPN class. Therefore, as we discussed in class, a model could just achieve a base 99.96% accuracy just by classifying everything as non-VPN. This severe class imbalance is most likely the reason behind these poor results. In order to get more meaningful results, we would need to either get more VPN data or perhaps try out a different model approach, like one-class SVM or an approach that better deals with extremely rare events (which, in this dataset, is what a VPN datapoint is).

In conclusion, the class imbalance of non-VPN and VPN samples inhibited the models from learning the differences between the two classes. As a result, the models achieve high accuracy but poor precision, indicating that when given new unseen data, they're likely to incorrectly label VPN data as non-VPN data. We used only 15 features for classification: three features measuring the time between packets (Inter-Arrival Time or IAT) and twelve features measuring packet sizes (STATS). These simple network-level features are effective because VPN encryption creates predictable patterns in timing and packet sizes that differ from regular traffic. Although we were able to pass the nPrint benchmark of 81.9% accuracy across all our models, we do not believe we've truly reached the benchmark, as our labeling and resulting class imbalance could have differed from the benchmark's.

Our work has important limitations. First, we only tested on Tor traffic, not commercial VPNs. Second, the dataset is extremely imbalanced, with VPN flows being only an incredibly small minority of the data. Third, these models were trained and tested in the same time period, so we cannot guarantee they will work on future traffic as VPN technology evolves (i.e., model drift). For real-world deployment, organizations should not rely on a single detection method. Combine VPN detection with other security signals like DNS queries, connection patterns, and behavioral analysis. Also, retrain models regularly as VPN techniques change and new applications emerge.

Works Cited

[1] https://nprint.github.io/benchmarks/application_identification/netml_vpn-nonvpn.html

[2]https://itp.nyu.edu/networks/explanations/demystifying-the-dark-web-an-introduction-to-tor-and-onion-routing/

Figures

Figure #1: *Random Forest Confusion Matrix*

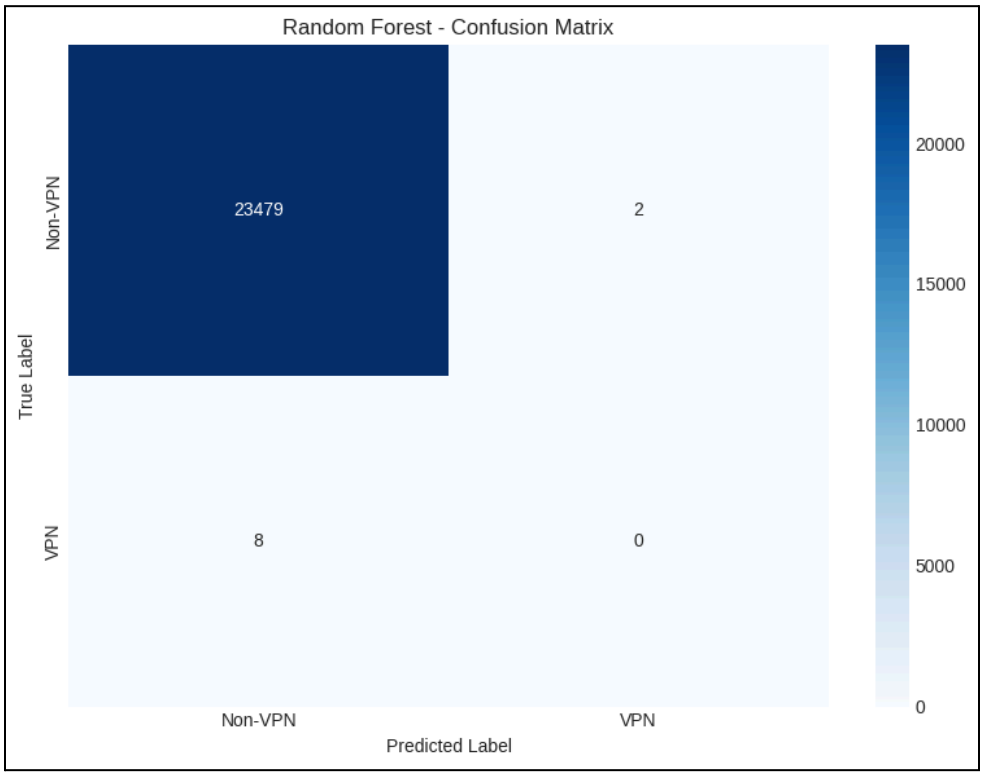

Figure #2: *Naive Bayes Confusion Matrix*

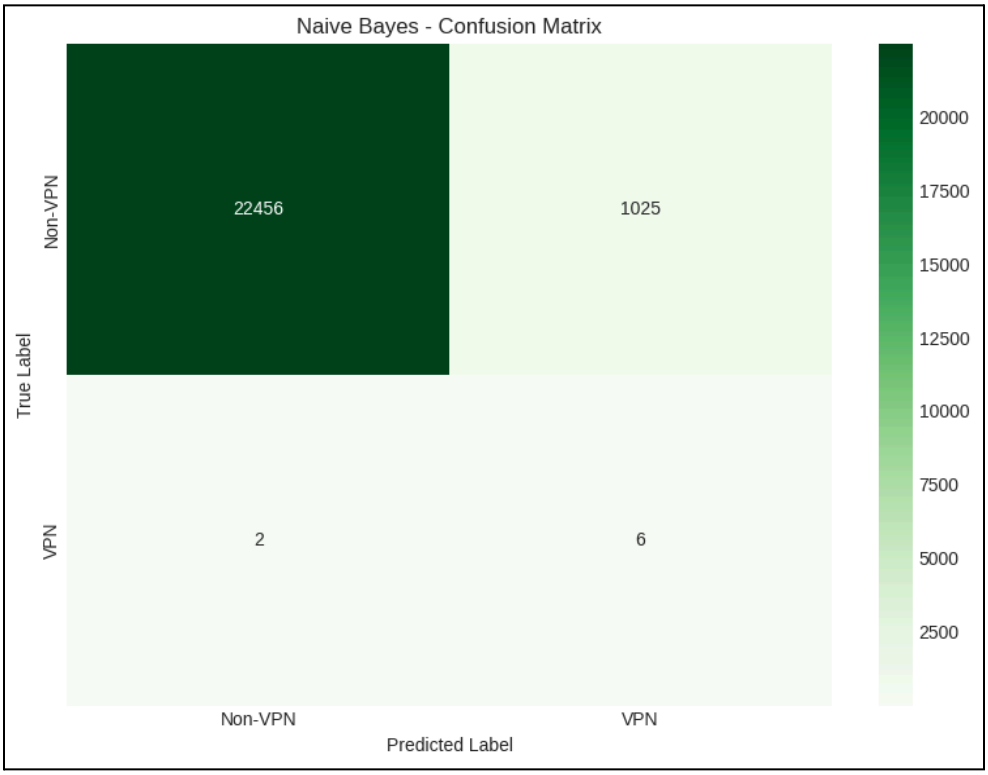

Figure #3: *Lasso Regression Confusion Matrix*

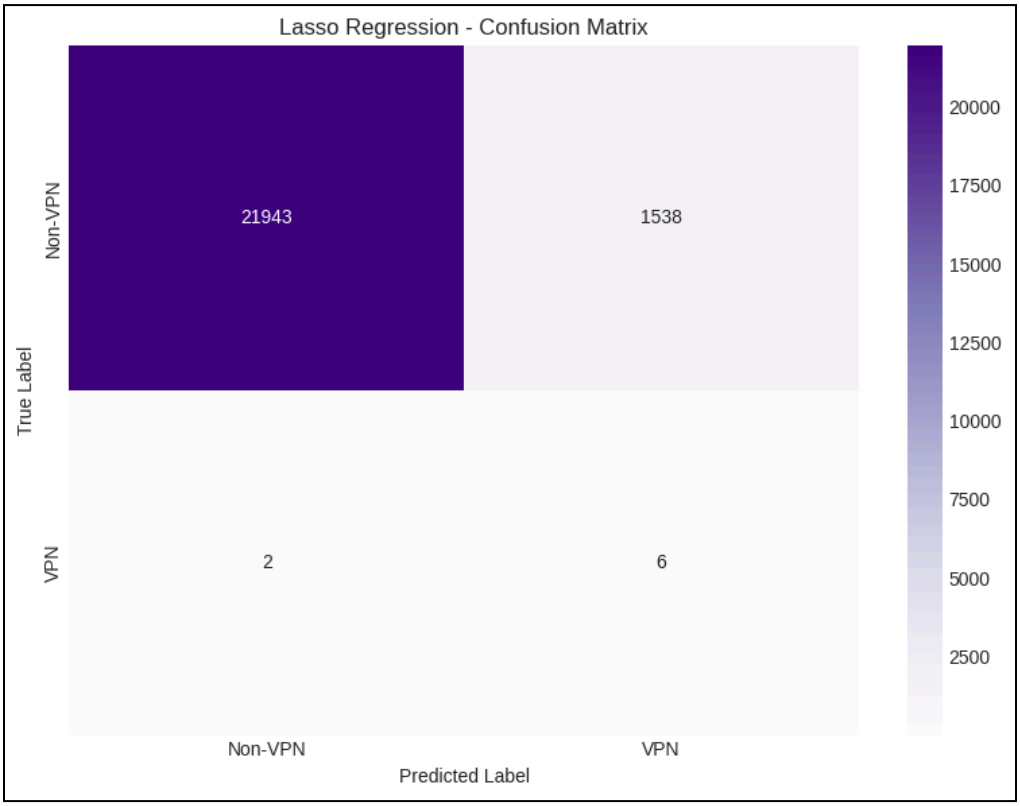

Figure #4: *Model Performance Comparisons (A).* Graphs comparing each model's accuracy, precision, recall, f1-score, ROC-AUC, and ROC curves side by side.

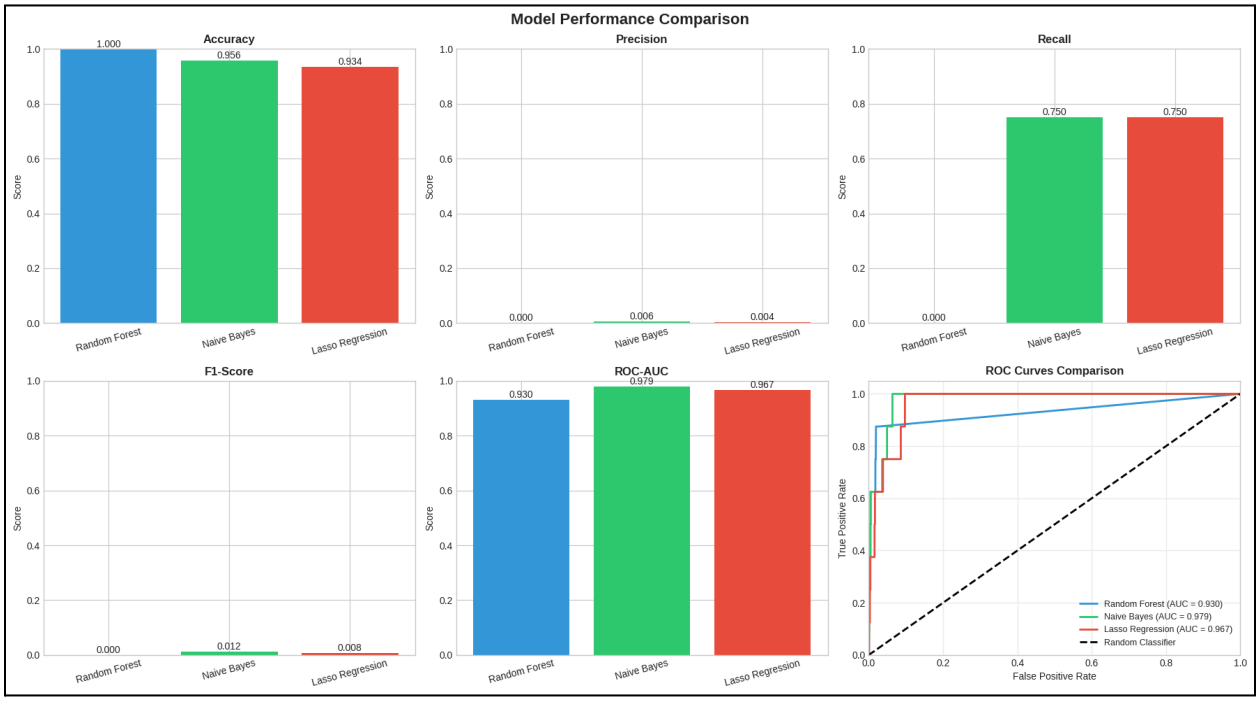

Figure #5: *Model Performance Comparisons (B).* Histograms of Accuracy, Precision, Recall, F1 score, and ROC-AUC scores of all three models.